CSE 4111
Introduction to Data Science
Chapter 2: Python for Data Science

COURSE COORDINATOR

MD. HASIBUL ISLAM
SENIOR LECTURER
CSE, NUBT KHULNA

Outlines

- Installation
- Command Line
- Indentation
- Comments
- Variables
- Data Type
- If .. Else
- Match
- Loops
- Function

Python Programming Language

- Introduced in 1991, now one of the most popular interpreted languages
- Widely used alongside Perl, Ruby
- Popular for web development (e.g., Django, Rails)

Key Characteristics

- Easy to use for quick scripts & automation
- Also powerful for serious software development
- Strong scientific computing & data analysis community

Python Programming Language

Growth & Importance

- Evolved into a leading language for:
  - Data Science
  - Machine Learning
  - Software Development

Why Python for Data Analysis

- Rich libraries like pandas, scikit-learn
- Competes with tools like R, MATLAB, SAS, Stata
- Suitable for both analysis and full application development

A versatile language for data, research, and real-world applications

Python: Installation

If Python is not installed on your computer, you can download it for free from the official website: https://www.python.org/

Check python installation

- Open Command Promt and Try: `C:\Users\YourName>py --version`

First Python program

Let's write our first Python file, called hello.py, which can be done in any text editor:

`hello.py`:

```
print("Hello, World!")
```

Simple as that. Save your file. Open your command line, navigate to the directory where you saved your file, and run: `C:\Users\YourName>py hello.py`

Output: `Hello, World!`

Command Line

Type the following on the Windows, Mac or Linux command line: `C:\Users\YourName>py`

From there you can write any python code, including our hello world example from earlier: `C:\Users\YourName>py`

```
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC
v.1900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> print("Hello, World!")
Hello, World!
```

Command Line

Whenever you are done in the python command line, you can simply type the following to quit the python command line interface: `exit()`

Command Line

IPython:

Installation:

Check pip:

```
C:\Users\HP>py -m pip --version
pip 26.1.1 from C:\Users\HP\AppData\Local\Programs\Python\Python314\Lib\site-packages\pip (python 3.14)
```

Command Line

If pip not available then install pip.

```
C:\Users\HP> py -m ensurepip --upgrade
```

Then install IPython:

```
C:\Users\HP>py -m pip install ipython
```

Run Ipython:

```
C:\Users\HP>py -m IPython
```

Command Line

IPython Example:

```
C:\Users\HP>py -m IPython
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 9.14.0 -- An enhanced Interactive Python. Type '?' for help.
Tip: You can change the editing mode of IPython to behave more like vi, or emacs.

In [1]: x=4

In [2]: y=8

In [3]: z=y/x*x

In [4]: z
Out[4]: 8.0

In [5]: z=y/(x*x)

In [6]: z
Out[6]: 0.5

In [7]: exit()
```

Indentation

Indentation refers to the spaces at the beginning of a code line.

Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.

Python uses indentation to indicate a block of code.

```
if 5 > 2:
  print("Five is greater than two!")
```

Indentation

Python will give you an error if you skip the indentation:

Syntax Error:

```
if 5 > 2:

print("Five is greater than two!")
```

Indentation

Statement 1:

```
if 5 > 2:
 print("Five is greater than two!")
if 5 > 2:
        print("Five is greater than two!")
```

Statement 2:

```
if 5 > 2:
 print("Five is greater than two!")
        print("Five is greater than two!")
```

Which one is right?

Comments

Single Line Comment:

```
#This is a comment
print("Hello, World!")
print("Hello, World!") #This is a comment
```

Multiline Comment:

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

Variables

Python has no command for declaring a variable.

A variable is created the moment you first assign a value to it.

```
x = 5
y = "John"
print(x)
print(y)
```

Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

```
x = 4       # x is of type int
x = "Sally" # x is now of type str
print(x)
```

Variables

If you want to specify the data type of a variable, this can be done with casting.

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

Strings:

```
x = "John"
# is the same as
x = 'John'
```

Variable Name

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

Python Keywords

and	A logical operator
as	To create an alias
assert	For debugging
async	Define an asynchronous function
await	Wait for and get a result from an awaitable
break	To break out of a loop
case	Pattern in a match statement

class	To define a class
continue	To continue to the next iteration of a loop
def	To define a function
del	To delete an object
elif	Used in conditional statements, same as else if
else	Used in conditional statements
except	Used with exceptions, what to do when an exception occurs

Python Keywords

False	Boolean value, result of comparison operations
finally	Used with exceptions, a block of code that will be executed no matter if there is an exception or not
for	To create a for loop
from	To import specific parts of a module
global	To declare a global variable
if	To make a conditional statement
import	To import a module

in	To check if a value is present in a list, tuple, etc.
is	To test if two variables are equal
lambda	To create an anonymous function
match	Start a match statement (compare a value against cases)
None	Represents a null value
nonlocal	To declare a non-local variable
not	A logical operator

Python Keywords

or	A logical operator
pass	A null statement, a statement that will do nothing
raise	To raise an exception
return	To exit a function and return a value
True	Boolean value, result of comparison operations
try	To make a try...except statement
while	To create a while loop
with	Used to simplify exception handling
yield	To return a list of values from a generator

Variable Name

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
2myvar = "John"
my-var = "John"
my var = "John"
```

Find Valid variables.

Data Types

Text Type:
`str`
Numeric Types:
`int`, `float`, `complex`
Sequence Types:
`list`, `tuple`, `range`
Mapping Type:
`dict`

Set Types:
`set`, `frozenset`
Boolean Type:
`bool`
Binary Types:
`bytes`, `bytearray`, `memoryview`
None Type:
`NoneType`

To get the data type:

```
x = 5
print(type(x))
```

Data Types

Complex (`complex`)

Description

Stores complex numbers.

Example

```
z = 3 + 4j
```

Use Cases

- Signal processing
- Electrical engineering
- Scientific computing

Data Types

Sequence Type (`list`)

Description

An ordered, mutable collection.

Example

```
fruits = ["Apple", "Banana", "Mango"]
```

Use Cases

- Storing multiple values
- Data processing
- Machine learning datasets

Key Features

- Mutable
- Allows duplicates

Data Types

Sequence Type (`tuple`)

Description

An ordered, immutable collection.

Example

```
point = (10, 20)
```

Use Cases

- Coordinates
- Fixed configuration values
- Returning multiple values from functions

Key Features

- Immutable
- Faster than lists

Data Types

Sequence Type (`range`)

Description

Represents a sequence of numbers.

Example

```
numbers = range(1, 6)
```

Output

1 2 3 4 5

Use Cases

- Loops
- Iteration
- Generating sequences

Data Types

Mapping Type (`dict`)

Description

Stores data as key-value pairs.

Example

```
student = {
    "name": "Ayat",
    "cgpa": 3.94
}
```

Use Cases

- Databases
- JSON data
- Configuration settings

Key Features

- Mutable
- Fast lookup by key

Data Types

Sequence Type (`range`)

Description

Represents a sequence of numbers.

Example

```
numbers = range(1, 6)
```

Output

1 2 3 4 5

Use Cases

- Loops
- Iteration
- Generating sequences

Data Types

Set Type (`set`)

Description

An unordered collection of unique elements.

Example

```
numbers = {1, 2, 3, 4}
```

Use Cases

- Removing duplicates
- Membership testing
- Mathematical set operations

Example Operations

A = {1,2,3}
B = {3,4,5}

A.union(B)
A.intersection(B)

Data Types

Set Type (`frozenset`)

Description

An immutable version of a set.

Example

```
fs = frozenset([1, 2, 3])
```

Use Cases

- Dictionary keys
- Read-only sets
- Security-sensitive data

Key Features

- Immutable
- Hashable

Data Types

Boolean Type (`bool`)

Description

Represents logical values.

Values

True
False

Example

```
is_student = True
```

Use Cases

- Conditions
- Decision making
- Control flow

Example

```
if is_student:
    print("Welcome")
```

Data Types

Binary Type (`bytes`)

Description

Immutable sequence of bytes.

Example

data = b"Hello"

Use Cases

- Network communication
- File handling
- Encryption

Data Types

Binary Type (`bytearray`)

Description

Mutable sequence of bytes.

Example

```
data = bytearray(b"Hello")
data[0] = 74
```

Output

bytearray(b'Jello')

Use Cases

- Buffer manipulation
- Image processing
- Streaming data

Data Types

Binary Type (`memoryview`)

Description

Provides direct memory access without copying data.

Example

```
data = bytes([1,2,3,4])
view = memoryview(data)
```

Use Cases

- Large datasets
- High-performance computing
- Efficient data processing

Benefit

Avoids unnecessary memory duplication.

Data Types

Binary Type (`bytearray`)

Description

Mutable sequence of bytes.

Example

```
data = bytearray(b"Hello")
data[0] = 74
```

Output

bytearray(b'Jello')

Use Cases

- Buffer manipulation
- Image processing
- Streaming data

Data Types

None Type (NoneType)

Description

Represents the absence of a value.

Example

result = None

Use Cases

- Default function arguments
- Missing values
- Placeholder variables

Example

```
def find_student(id):
    return None
```

Key Feature

Only one value exists:

None

If...Else

If:

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

Elif

Elif:

```
a = 33
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
```

If...elif...else

Else:

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```

Match

```
day = 4
match day:
  case 1:
    print("Monday")
  case 2:
    print("Tuesday")
  case 3:
    print("Wednesday")
  case 4:
    print("Thursday")
  case 5:
    print("Friday")
  case 6:
    print("Saturday")
  case 7:
    print("Sunday")
```

Match

```
day = 4
match day:
  case 6:
    print("Today is Saturday")
  case 7:
    print("Today is Sunday")
  case _:
    print("Looking forward to the Weekend")
```

while Loop

```
i = 1
while i < 6:
  print(i)
  i += 1
```

Break:

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

Continue:

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

else:

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

for Loop

Looping through a list:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

Looping through a string:

```
for x in "banana":
  print(x)
```

Continue:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    continue
  print(x)
```

The range() function:

```
for x in range(6):
  print(x)
```

type 2:

```
for x in range(2, 6):
  print(x)
```

Type 3:

```
for x in range(2, 30, 3):
  print(x)
```

for Loop

Else in for loop:

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```

Function

```
def fahrenheit_to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5 / 9
```

Thank you

for your Attention